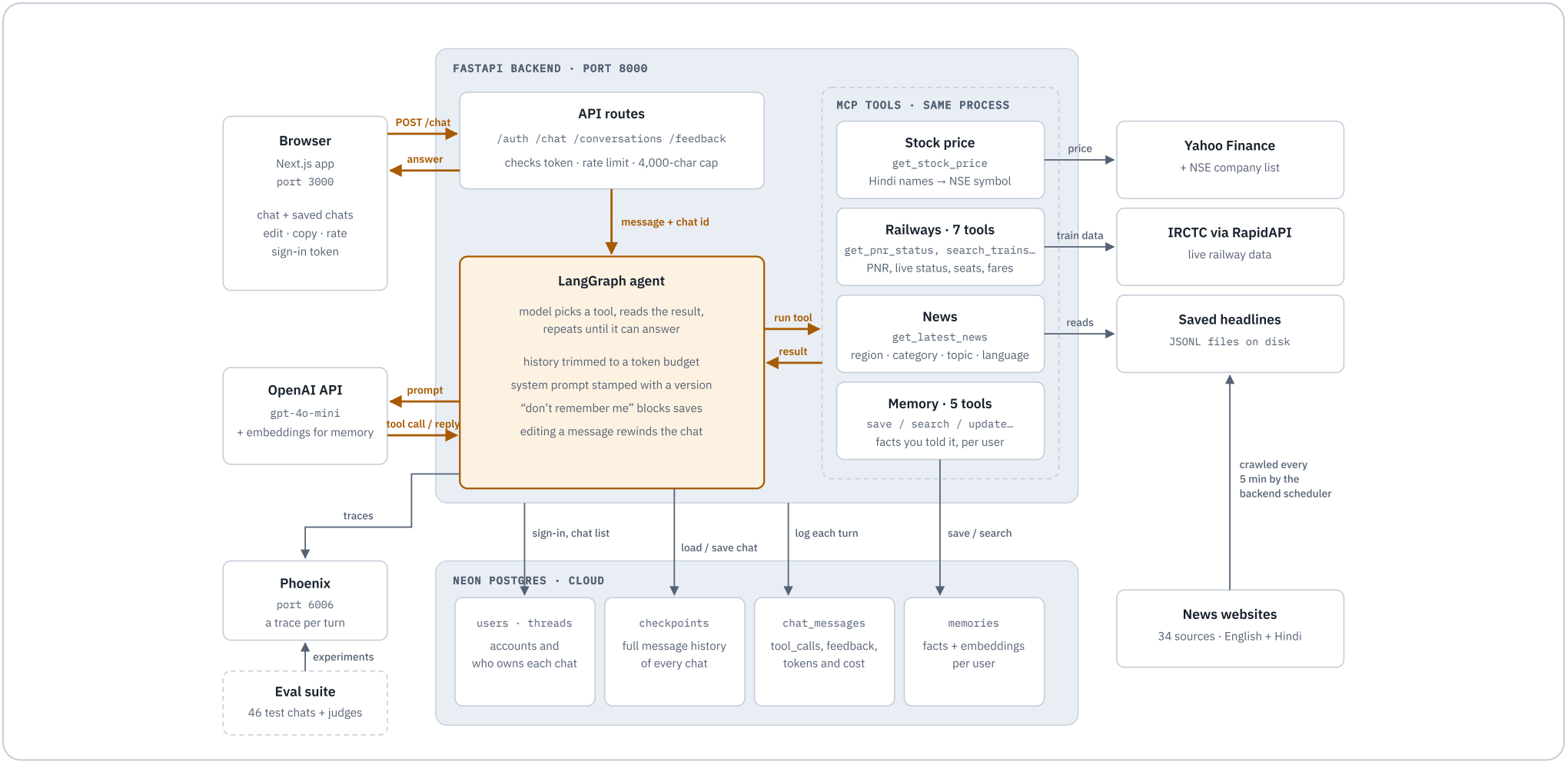
VOXPATH · SYSTEM MAP

How VoxPath answers a question

A Next.js chat app talks to one FastAPI backend. Inside it, a LangGraph agent lets gpt-4o-mini choose among 14 tools for stocks, trains, news and memory, then answers from what the tools return.



Saffron arrows are the path of one question, from the browser to the model and tools and back. Grey arrows are the reads and writes around it. The tools run inside the backend process; only the boxes on the far right and OpenAI are outside services.

One question, step by step

1. **You ask** “top political news from Bihar”. The browser posts it to `/chat` with your sign-in token and the chat’s id.
2. **The backend checks** the token, your rate limit and the message length, and records that this chat is yours.
3. **The agent loads the chat** from Postgres and sends it, with the system prompt, to gpt-4o-mini.
4. **The model picks a tool**: `get_latest_news(region="Bihar", category="politics")`.
5. **The news tool searches** the headlines the crawler saved in the last 72 hours and returns five, each with its link.
6. **The model writes the answer** from that result, and the agent saves the updated chat.
7. **The backend replies** and logs the turn (tools, tokens, cost, time). Phoenix receives the whole trace.

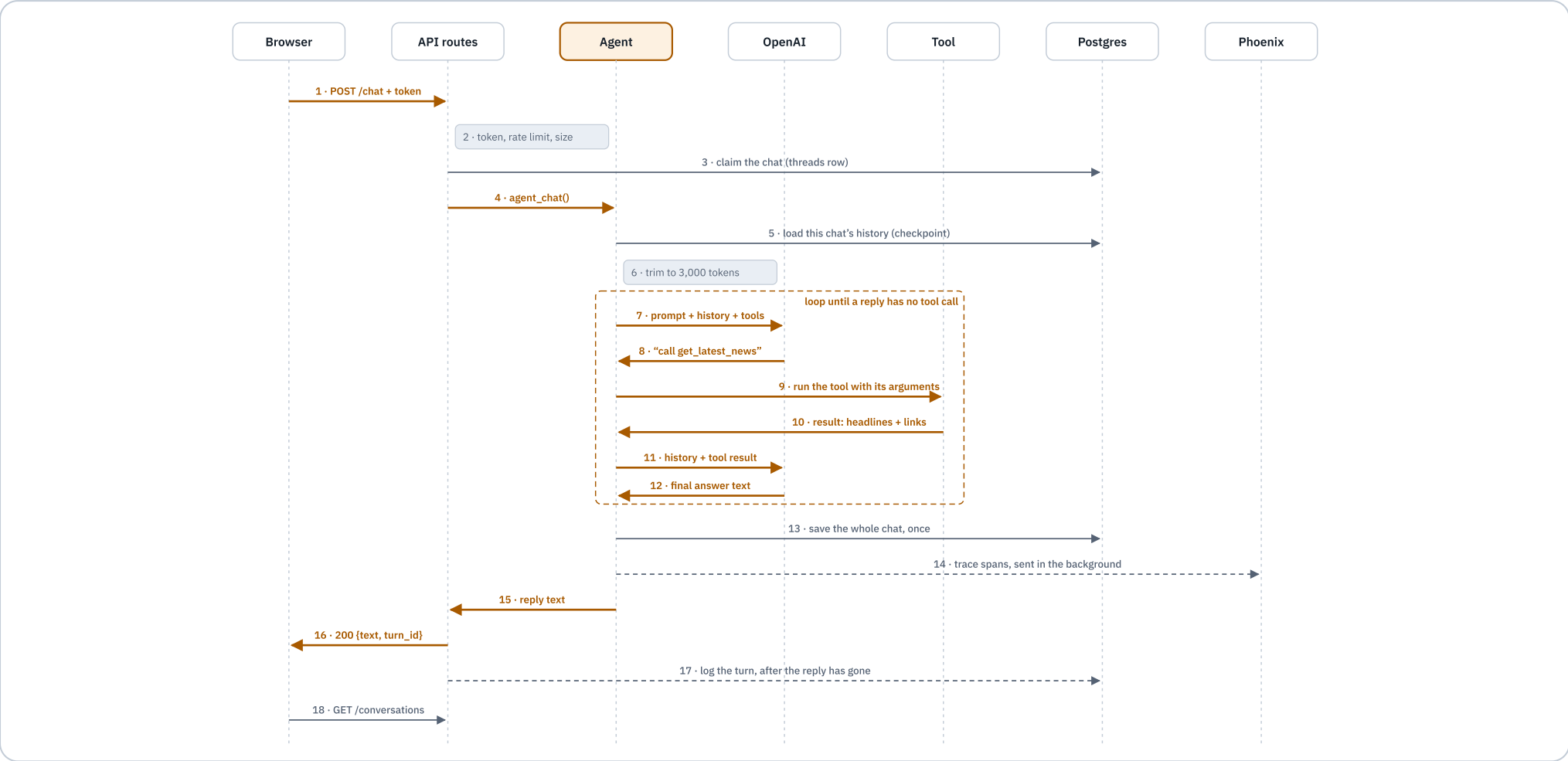
Where things are kept

WHAT	WHERE
Chats	Postgres checkpoints; the sidebar lists them from `threads`
Your facts	Postgres memory store, searched by meaning with embeddings
Headlines	JSONL files on the backend’s disk, refreshed every 5 minutes
Turn logs	`chat_messages`, `tool_calls`, `message_feedback`
Traces	Phoenix’s own database on your machine (`tracing/data`)
Test chats	`backend/evals/cases.jsonl`; results in `evals/results`

HOP BY HOP

The full route of one message

Every call your question causes, in the order it happens. The numbers in the diagram match the stages below, which name the code that does each step.



Saffron is your question and its answer. Grey is bookkeeping around it; dashed grey happens in the background and never makes you wait. Steps 7 to 12 repeat once per tool round: a news question takes one round, a railway fare usually takes two (find the station codes, then the fare).

STEP 1

In the browser

When you press Enter, `sendTextMessage` puts your message on screen at once, then calls `authFetch('/chat')`. That adds your sign-in token as an `Authorization: Bearer` header. The body carries the message and the chat's id: a new chat gets a fresh UUID on its first message, and an edit also sends `edit_index` and `edit_original`.

frontend/src/components/ChatInterface.tsx · sendTextMessage
frontend/src/lib/auth.ts · authFetch

STEPS 2–3

Through the backend's front door

CORS lets only the app at `localhost:3000` call the API from a browser. A middleware gives the request a log id and times it. FastAPI then runs the route's checks in order:

- `current_user` verifies the token's signature with `JWT_SECRET` and gives the user. This is the only place a user id comes from; nothing the browser sends can claim to be someone else.
- `limit_chat` allows 30 chats a minute per user.
- `chat_endpoint` refuses an empty message, one over 4,000 characters, or a malformed chat id.
- `claim_thread` writes one `threads` row in a single SQL statement: it claims a new chat for you, or touches one you own. A chat owned by someone else gets 404, so its existence isn't even confirmed.

backend/main.py · trace_requests, chat_endpoint
backend/auth.py · current_user, claim_thread
backend/ratelimit.py · limit_chat

STEPS 4–6

Into the agent

`chat_endpoint` opens a turn record, which collects every tool call and token count, and calls `agent_chat`. That builds a config with two ids: `thread_id` says which chat, `user_id` says whose memories. It opens the Phoenix trace with your session, user, prompt version and model, then hands your message to the LangGraph agent.

LangGraph loads the chat's checkpoint from Postgres and adds your message. Before every model call, the pre-model hook:

- applies any "don't remember me" request found anywhere in the chat, so memory saves are refused for the whole turn;
- shortens tool results over 6,000 characters;
- keeps the newest messages that fit in 3,000 tokens, always starting at one of your messages so no tool result loses its call.

Only what the model sees is trimmed. The saved chat keeps everything.

backend/agent.py · `agent_chat, _build_pre_model_hook, record_turn`

STEPS 7–12

The model-and-tools loop

The agent is a graph with two nodes. The *model* node sends the system prompt, the trimmed history and the descriptions of all 14 tools to gpt-4o-mini. If the reply asks for tools, the *tools* node runs each one, adds its result to the history, and the model node runs again. The first reply without a tool call is the answer.

No code routes by keywords. The model reads the system prompt and the tool descriptions and picks the tool and its arguments; that is why the Bihar fix was a change to the description and the prompt. Each tool is wrapped so its arguments, result and time are logged and added to the turn record.

- **Stocks.** `get_stock_price` gets the name as you typed it. Hindi is turned into Latin letters and matched by sound against the NSE company list; Yahoo Finance gives the price. The answer names the company, so a near-match is visible.
- **Railways.** City names go through `resolve_station_code` first. The other tools call IRCTC through RapidAPI; `search_trains` asks several station pairs at once and returns whatever arrived before its deadline.
- **News.** `get_latest_news` searches headlines already saved on disk, so no website is fetched during your chat. It filters by region, category, topic and language; if nothing matches, it relaxes one filter at a time and adds a note saying what wasn't found.
- **Memory.** The five memory tools read your user id from the config, never from the model, so one user can't reach another's memories. Search works by meaning, using OpenAI embeddings stored in Postgres.

backend/agent.py · _build_agent, _with_call_logging
backend/mcp_server.py · tool definitions
backend/services.py, **stock_resolver.py**, **railway.py**, **news_crawler.py** · search_news, **memory.py**

STEPS 13–17

Saving and replying

When the graph finishes, LangGraph saves the whole chat to Postgres in one write. Saving after every step used to add about 2 seconds to each answer. `agent_chat` adds the token counts and the Phoenix trace id to the turn record and returns the answer text.

`chat_endpoint` replies `200` with the text and a `turn_id`. Only after the reply has gone does a background task write the logs: one `chat_messages` row with the question, answer, tools, tokens, cost, time, prompt version and trace id, and one `tool_calls` row per tool. Phoenix spans leave in batches in the background too, so a slow or stopped Phoenix never slows a chat.

backend/agent.py · agent_chat, _note_usage
backend/main.py · recorded
backend/analytics.py · record_turn
backend/tracing.py · setup_tracing

STEPS 16, 18

Back in the browser

The answer appears as Markdown, with links opening in a new tab and Copy and thumbs buttons under it. The page then asks `GET /conversations`, so a new chat appears in the sidebar and the open one moves to the top. A thumbs click posts `/feedback` with the `turn_id`; that row links your rating to the logged turn and its Phoenix trace.

frontend/src/components/ChatInterface.tsx · sendTextMessage, rateAnswer
backend/conversations.py · list and open chats

When a step fails

WHAT HAPPENED	REPLY	WHAT YOU SEE
No token, or it expired	401	You are signed out and asked to sign in again
More than 30 chats in a minute	429	“Too many requests. Please wait N seconds”
Message over 4,000 characters, or a bad chat id	400	The reason, under the input box
Someone else’s chat, or one you hid	404	The chat leaves your list and a new one opens
The message you edited changed meanwhile	409	“Reopen the chat and try again”; nothing was changed
Database unreachable before the agent ran	503	“Reconnecting”, then one automatic resend
Database unreachable after the agent ran	503	Asked to retry yourself, since tools may already have run
OpenAI quota or rate limit	200	A short fallback answer; the turn is logged as an error
Anything else	500	“Something went wrong… (Reference: turn id)”; the full error is in the log and analytics
Backend not running	none	“Connection Error”, with the address it tried